

"Hola Mundo" Orientado a Objetos en C++

1. Objetivo de la práctica

Construir un primer programa orientado a objetos en C++ que modele dos objetos de una misma clase que se comunican entre sí mediante el envío de mensajes (llamadas a métodos), aplicando de forma práctica los conceptos revisados en la teoría de la Unidad I.

Al finalizar esta práctica, el estudiante habrá:

- Definido una clase con un atributo y un método.
 - Creado (instanciado) dos objetos de esa clase.
 - Implementado un método que recibe un objeto por referencia y lee su estado.
 - Comprendido, en código, qué significa que "un objeto envía un mensaje a otro".
-

2. Paso a paso guiado

Paso 1 — Definir la clase

Toda clase representa un molde a partir del cual se crean objetos. En este caso, vamos a modelar el concepto de `Persona`.

```
class Persona {  
  
};
```

Nota teórica — Clase: una clase es la plantilla que define qué atributos (estado) y qué métodos (comportamiento) tendrán los objetos creados a partir de ella. No ocupa memoria de "instancia" hasta que se crea un objeto.

Paso 2 — Definir el atributo `nombre`

Dentro de la clase, agregamos el atributo que representará el estado de cada objeto `Persona`.

```
class Persona {  
    private:  
        string nombre;  
};
```

Nota teórica — Atributo: un atributo es una variable que pertenece a la clase y representa parte del estado de cada objeto. Cada instancia de `Persona` tendrá su propia copia de `nombre`, con un valor independiente.

Nota teórica — `private`: declarar el atributo como `private` significa que solo es accesible desde dentro de la propia clase. Esto es la base del **encapsulamiento**: el estado interno del objeto no se expone directamente hacia afuera.

Paso 3 — Agregar el constructor

Necesitamos una forma de inicializar el atributo `nombre` en el momento en que se crea cada objeto.

```
class Persona {
    private:
        string nombre;

    public:
        Persona(string n) {
            nombre = n;
        }
};
```

Nota teórica — Constructor: un constructor es un método especial que se ejecuta automáticamente al crear un objeto (instanciarlo). Su propósito es dejar el objeto en un estado válido e inicializado desde el momento cero, evitando atributos sin valor definido.

Nota teórica — `public`: a diferencia de `nombre`, el constructor se declara `public` porque necesita ser accesible desde fuera de la clase (por ejemplo, desde `main()`) para poder crear objetos.

Paso 4 — Agregar un método de acceso (`getNombre`)

Como `nombre` es `private`, necesitamos un método público que permita consultarlo de forma controlada.

```
class Persona {
    private:
        string nombre;

    public:
        Persona(string n) {
            nombre = n;
        }

        string getNombre() {
            return nombre;
        }
};
```

Nota teórica — Método de acceso (getter): un getter es un método público que expone, de forma controlada, el valor de un atributo privado. Es un patrón fundamental del encapsulamiento: el mundo exterior no accede al atributo directamente, sino a través de una interfaz definida por la clase.

Paso 5 — Definir el método `saludar`

Este es el método central de la práctica: un objeto `Persona` va a comunicarse con otro objeto `Persona` mediante el envío de un mensaje.

```
class Persona {
private:
    string nombre;

public:
    Persona(string n) {
        nombre = n;
    }

    string getNombre() {
        return nombre;
    }

    void saludar(Persona &otraPersona) {
        cout << "Hola " << otraPersona.getNombre()
            << ", mi nombre es " << nombre << endl;
    }
};
```

Nota teórica — Método: un método es una función que pertenece a la clase y representa el comportamiento de sus objetos. En este caso, `saludar()` define la acción "saludar" que cualquier objeto `Persona` puede ejecutar.

Nota teórica — Parámetro por referencia (`Persona &otraPersona`): al recibir el parámetro como referencia (&), el método no crea una copia del objeto recibido; trabaja directamente sobre el objeto original en memoria. Esto es más eficiente y es la forma común de pasar objetos entre métodos en C++.

Nota teórica — Mensaje entre objetos: cuando el objeto que ejecuta `saludar()` llama a `otraPersona.getNombre()`, está enviando un mensaje al objeto recibido como parámetro. Este es exactamente el mecanismo de comunicación entre objetos que vimos en la teoría: un objeto invoca un método de otro objeto para obtener información o provocar una acción.

Paso 6 — Crear los objetos (instanciación) en `main()`

Con la clase completa, ahora creamos dos objetos (instancias) de tipo `Persona`.

```
int main() {
    Persona alumno1("Roberto");
    Persona alumno2("Ana");

    return 0;
}
```

Nota teórica — Instancia / Objeto: cada vez que se escribe `Persona alumno1("Roberto");`, se está creando una instancia (objeto) de la clase `Persona`. `alumno1` y `alumno2` son dos objetos distintos, cada uno con su propio valor de `nombre`, aunque ambos comparten la misma clase (el mismo molde).

Paso 7 — Enviar el mensaje `saludar`

Finalmente, hacemos que los objetos se comuniquen entre sí.

```
int main() {
    Persona alumno1("Roberto");
    Persona alumno2("Ana");

    alumno1.saludar(alumno2);
    alumno2.saludar(alumno1);

    return 0;
}
```

Nota teórica — Invocación de método: `alumno1.saludar(alumno2)`; es la sintaxis con la que un objeto (`alumno1`) ejecuta uno de sus propios métodos, enviándole como parámetro una referencia a otro objeto (`alumno2`). Este es el punto donde toda la teoría de la Unidad I se materializa en código funcional.

3. Programa completo

```
#include <iostream>
#include <string>
using namespace std;

class Persona {
private:
    string nombre;

public:
    Persona(string n) {
        nombre = n;
    }

    string getNombre() {
        return nombre;
    }

    void saludar(Persona &otraPersona) {
        cout << "Hola " << otraPersona.getNombre()
              << ", mi nombre es " << nombre << endl;
    }
};

int main() {
    Persona alumno1("Roberto");
    Persona alumno2("Ana");

    alumno1.saludar(alumno2);
    alumno2.saludar(alumno1);

    return 0;
}
```

Salida esperada:

Hola Ana, mi nombre es Roberto
Hola Roberto, mi nombre es Ana

4. Explicación del programa desde la perspectiva de la Orientación a Objetos

Este programa, aunque breve, materializa los conceptos fundamentales de la Unidad I:

- **Clase (`Persona`):** actúa como el molde que define qué atributos (`nombre`) y qué comportamientos (`saludar()`, `getNombre()`) tendrá cualquier objeto de tipo `Persona`. No representa a ninguna persona en particular, sino la estructura común a todas.
- **Objetos (`alumno1`, `alumno2`):** son instancias concretas de la clase `Persona`. Cada uno posee su propio estado independiente (`nombre = "Roberto"` y `nombre = "Ana"`, respectivamente), aunque ambos comparten el mismo comportamiento definido por la clase.
- **Encapsulamiento:** el atributo `nombre` está oculto (`private`) y solo es accesible mediante el método público `getNombre()`. Ningún código externo a la clase puede modificar o leer `nombre` directamente; debe hacerlo a través de la interfaz que la clase decide exponer.
- **Mensajes entre objetos:** la línea `alumno1.saludar(alumno2)`; representa el envío de un mensaje del objeto `alumno1` hacia sí mismo (invocando su propio método), y dentro de ese método, `alumno1` envía a su vez un mensaje al objeto `otraPersona` (que en tiempo de ejecución es `alumno2`) al invocar `otraPersona.getNombre()`. Esto ilustra cómo los objetos colaboran entre sí para producir el comportamiento del sistema, en lugar de que una función externa manipule los datos de ambos directamente.
- **Paso por referencia:** al recibir `otraPersona` por referencia (`&`), el método `saludar()` no trabaja sobre una copia del objeto, sino sobre el objeto real en memoria. Esto refuerza la idea de que, en un sistema orientado a objetos, los objetos interactúan entre sí como entidades con identidad propia, no como datos duplicados.
- **Constructor:** garantiza que ningún objeto `Persona` pueda existir sin un `nombre` definido, evitando estados inválidos o inconsistentes desde el momento de la creación del objeto.

En conjunto, el programa demuestra el cambio de paradigma central de la POO: en lugar de tener datos sueltos (`string nombre1, nombre2;`) manipulados por funciones externas, se tienen **objetos autónomos que encapsulan su propio estado y exponen comportamientos**, comunicándose entre sí mediante el envío de mensajes — que en C++ se traduce en la invocación de métodos.

5. Sugerencia de cierre de práctica

- Pedir a cada equipo que extienda el programa agregando un tercer objeto `Persona` y haciendo que los tres se saluden entre sí.
- Como reto opcional, proponer que el método `saludar()` también reciba un segundo parámetro (por ejemplo, un mensaje personalizado) para reforzar el manejo de parámetros en métodos.

- Recordar que este mismo patrón (atributo privado + getter + método que recibe un objeto por referencia) se reutilizará constantemente en los proyectos parciales del semestre.

Reto opcional — `responderSaludo()`

Nivel: opcional, para equipos que terminen antes de tiempo o quieran profundizar. **Nota para el docente:** este reto no se explica formalmente en la práctica de hoy; se retomará y formalizará en una sesión posterior. El objetivo es solo despertar la curiosidad sobre el tema.

Agregar a la clase `Persona` un nuevo método:

```
void responderSaludo(Persona &otraPersona) {  
    cout << otraPersona.getNombre() << ", el gusto es mío. Saludos, "  
        << nombre << endl;  
}
```

Y modificar `main()` para usarlo:

```
int main() {  
    Persona alumno1("Roberto");  
    Persona alumno2("Ana");  
  
    alumno1.saludar(alumno2);  
    alumno2.responderSaludo(alumno1);  
  
    return 0;  
}
```

Pregunta para reflexionar en equipo (no es necesario resolverla hoy):

¿Qué pasaría si `responderSaludo()` llamara internamente a `saludar()` en lugar de repetir la lógica del `cout`? ¿Sería lo mismo? ¿Qué significaría, en términos de mensajes entre objetos, que un método provoque la ejecución de otro método como reacción?

Esta pregunta no busca una respuesta cerrada hoy — es una semilla para un tema que retomaremos más adelante en el curso: **un mensaje recibido por un objeto puede, a su vez, desencadenar nuevos mensajes hacia otros objetos**, formando cadenas de colaboración entre objetos. Por ahora basta con que el equipo identifique que `responderSaludo()` es, en esencia, una reacción a haber recibido el mensaje `saludar()`.